API Testing Interview Guide

Top 50 questions with answers, e-commerce scenarios and hands-on practice steps

Written from a QA lead's point of view (10+ years). Each question has a clear answer, a real-time e-commerce example, and steps to practise it yourself, so you can explain it with confidence in an interview.

How to use this guide

- Read one section a day. Run the examples in Postman, REST Assured or k6 so you can talk from experience.
- Answer in four steps: define, give an e-commerce example, explain how you tested it, share a result or lesson.
- Where numbers appear (for example response-time targets), they are examples. Say that real values are agreed with the team.
- No list of 50 can predict every question an interviewer will ask. This guide covers the main areas (fundamentals, functional design, security, data, automation, performance, advanced). If you understand the reasoning, you can handle follow-ups.

Sample e-commerce API used in the examples

Area	Endpoints
Authentication	POST /auth/login, POST /auth/refresh, POST /auth/logout
Catalog	GET /products?q=&category=&sort=&page=&size=, GET /products/{id}
Cart	GET /cart, POST /cart/items, PATCH /cart/items/{id}, DELETE /cart/items/{id}
Coupons	POST /coupons/apply, DELETE /coupons/{code}
Orders	POST /orders, GET /orders/{id}, PATCH /orders/{id}/cancel, POST /returns
Payments	POST /payments, POST /webhooks/payment, POST /refunds
Admin	POST /admin/coupons, GET /admin/orders, PUT /admin/inventory/{id}

Contents

Section	Topic	Questions
Section A	API Fundamentals	Q1 to Q10
Section B	Functional Test Design (E-commerce Scenarios)	Q11 to Q20
Section C	Security and Authentication	Q21 to Q28
Section D	Data, Integration and Async Flows	Q29 to Q35
Section E	API Automation and CI/CD	Q36 to Q43
Section F	Performance and Resilience	Q44 to Q47
Section G	Advanced and Lead-Level Questions	Q48 to Q50
Extra	Universal API test checklist, how to answer, 60-second project pitch	Last pages

Section A: API Fundamentals

Q1. What is API testing, and how is it different from UI testing? Why does it matter in an e-commerce application?

Answer

API testing verifies the business logic, data, security and performance of the service layer by sending requests directly to endpoints and validating the response (status code, headers, body, response time) and the side effects (database, events, emails). UI testing drives the browser and checks what the user sees.

API tests are faster, more stable and run earlier. They can also reach inputs the UI never allows: negative quantities, tampered prices, missing fields, expired tokens and parallel requests.

Real-time e-commerce example

The checkout page only lets a user pick quantity 1 to 10 from a dropdown. At API level you send quantity -1, 0 and 9999 to POST /cart/items and confirm the server rejects them. A UI test can never send those values.

How to work on it

- Import the Swagger/OpenAPI file or Postman collection and call every endpoint manually first.
- Then add assertions for status code, schema, business rules and database state.
- Finally automate the stable cases and run them in CI.

Q2. Where does API testing fit in the test pyramid, and what do you mean by shift-left?

Answer

The pyramid has many unit tests at the bottom (developers), a medium layer of API/service tests in the middle (fast and stable), and a few UI end-to-end tests on top (slow and fragile). Most business-rule validation belongs in the API layer.

Shift-left means testing starts early. Once the API contract (OpenAPI) is agreed, QA can write test cases and use mock servers before the UI or even the backend is finished.

Real-time e-commerce example

Order total = items + tax + shipping - coupon. Instead of 40 slow UI runs, you run 40 API cases in seconds and keep only 2 UI tests to confirm the total is displayed correctly.

How to work on it

- Review your current suite and move rule-heavy UI checks down to the API layer.
- Keep UI tests for a few critical journeys (login, search, checkout).
- Join design reviews and read the API spec before development finishes.

Q3. What is the difference between REST, SOAP and GraphQL, and how does testing differ for each?

Answer

REST uses HTTP methods and usually JSON. SOAP uses an XML envelope with a WSDL contract and built-in standards such as WS-Security. GraphQL exposes one endpoint where the client asks for exactly the fields it needs.

Aspect	REST	SOAP	GraphQL
Format	JSON over HTTP verbs	XML envelope, WSDL	One endpoint, query language
Contract	OpenAPI / Swagger	WSDL / XSD	Schema (types, queries, mutations)
E-commerce use	Storefront and mobile APIs	Legacy ERP, bank or courier integrations	Flexible product and home pages
Testing focus	Methods, status codes, schema, auth	XML validation, SOAP faults, security headers	Queries and mutations, field-level auth, query depth limits

Real-time e-commerce example

GraphQL usually returns HTTP 200 even when a query fails, with an 'errors' array in the body. So a GraphQL test must assert the body, not just the status code.

How to work on it

- Practise REST with Postman first.
- Use SoapUI or Postman with an XML body to try a public WSDL.
- Try a public GraphQL API and write one query and one mutation test.

Q4. What are the main parts of an API request and response?

Answer

Request: method, URL (base URL, path, query string), headers, authentication and body. Response: status code, headers, body, cookies and response time. A good test checks all of them, not just the body.

```
POST /api/v1/orders HTTP/1.1
Host: qa.shop.example.com
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{ "items": [{ "productId": 101, "quantity": 2 }],
  "addressId": 55, "couponCode": "FEST10", "paymentMethod": "UPI" }

HTTP/1.1 201 Created
Location: /api/v1/orders/ORD-9002
Content-Type: application/json

{ "orderId": "ORD-9002", "status": "CREATED", "total": 1799.00 }
```

Real-time e-commerce example

For the order above you assert: status is 201, the Location header exists, Content-Type is JSON, orderId is present, status is CREATED, total matches the expected calculation, and the response time is within the limit.

How to work on it

- Send the request in Postman and open the Console to see the raw request and response.
- Write one assertion for each part: status, header, body field, response time.

Q5. Explain the HTTP methods. Which are safe and idempotent? What is the difference between PUT and PATCH?

Answer

Safe means the request does not change data. Idempotent means repeating the same request leaves the system in the same final state. PUT replaces the whole resource. PATCH changes only the fields you send.

Method	Purpose	Safe	Idempotent	E-commerce example
GET	Read	Yes	Yes	GET /products/101
POST	Create / action	No	No	POST /orders
PUT	Replace whole resource	No	Yes	PUT /users/55/address (full address)
PATCH	Partial update	No	Not guaranteed	PATCH /cart/items/9 {quantity: 3}
DELETE	Remove	No	Yes	DELETE /cart/items/9

Real-time e-commerce example

Send the same PUT address update twice: the final address must be identical. Send DELETE cart item twice: the second call returns 404 or 204 (per design) but must not change anything else. Send POST /orders twice without protection and you get two orders, which is why idempotency keys exist.

How to work on it

- For each endpoint in your collection, write down whether it should be idempotent and test it by calling it twice.
- Check that PUT with missing fields does not silently wipe data, and PATCH leaves other fields unchanged.

Q6. Which HTTP status codes do you expect in an e-commerce API, and when?

Answer

Assert both the status code and the error body (error code and message). Also confirm that errors never show stack traces.

Code	Meaning	E-commerce scenario
200	OK	Get product, get cart, search results
201	Created	Order placed (with a Location header)
204	No Content	Cart item deleted
400	Bad Request	Malformed JSON or missing required field
401	Unauthorized	Token missing, invalid or expired
403	Forbidden	Customer token calling an admin API
404	Not Found	Product ID does not exist
409	Conflict	Duplicate email on sign-up, item out of stock, stale version
422	Unprocessable Entity	Valid JSON but a business rule fails (expired coupon)
429	Too Many Requests	Rate limit hit (with a Retry-After header)
500	Server Error	Unhandled exception. Always log it as a bug
502/503/504	Gateway / Unavailable / Timeout	Payment gateway or downstream service down or slow

Real-time e-commerce example

Apply an expired coupon: the response should be 422 (or 400, per contract) with a clear message such as COUPON_EXPIRED, never a 500.

How to work on it

- Build a status-code matrix for every endpoint: which codes it can return and which test triggers each.
- Log any 500 as a defect, because a client mistake should never crash the server.

Q7. What is the difference between 401 and 403, between 400 and 422, and what should happen when a user requests someone else's order?

Answer

401 means the caller is not authenticated (no, invalid or expired credentials). 403 means the caller is authenticated but not allowed to do this.

400 usually means the request cannot be parsed or is structurally wrong. 422 means it is well formed but fails validation or business rules. Teams differ, so follow the contract and check for consistency.

When user A asks for user B's order, the server should return 403 or 404. Some teams prefer 404 so the existence of the order is not revealed.

Real-time e-commerce example

GET /orders/9002 with user A's token where the order belongs to user B must not return 200 with B's data.

How to work on it

- Test each endpoint with: no token (401), valid customer token on an admin endpoint (403), and another user's resource ID (403/404).
- Document the status-code rules agreed with the developers.

Q8. Which request and response headers matter most when testing APIs?

Answer

Content-Type / Accept: what format is sent and expected. **Authorization**: credentials. **Cache-Control / ETag / If-None-Match**: caching. **Idempotency-Key**: prevents duplicate orders and payments. **X-Request-ID / Correlation ID**: tracing in logs. **Location**: URL of a created resource. **Retry-After**: with 429/503.

Security headers: Strict-Transport-Security, X-Content-Type-Options, and cookie flags HttpOnly, Secure and SameSite. CORS headers: Access-Control-Allow-Origin.

Real-time e-commerce example

Send POST /orders with Content-Type: text/plain. The API should answer 415 Unsupported Media Type, not 500. Remove the Authorization header and expect 401.

How to work on it

- Add header checks to your Postman tests (pm.response.to.have.header('Content-Type')).
- Check cookies for the Secure and HttpOnly flags after login.

Q9. What is the difference between path parameters, query parameters and the request body? How do you test each?

Answer

Path parameters identify a resource (/products/101). Query parameters filter, sort or paginate (?category=shoes&page=2). The body carries the data for create and update calls.

For each one test: missing, empty, wrong type (page=abc), negative, very large, special characters, SQL characters, duplicate parameters and very long values.

Real-time e-commerce example

GET /products?page=-1&size=100000 should return a clean 400 (or a capped page size), not a 500 or a huge response. GET /products/abc should be 400 or 404.

How to work on it

- Make a small table per endpoint listing every parameter, whether it is required, its type and its limits.
- Turn each row into positive, negative and boundary tests.

Q10. What is an OpenAPI (Swagger) specification and an API contract? How do you use them in testing?

Answer

An OpenAPI file describes endpoints, parameters, request and response schemas, status codes and authentication. It is the contract between backend, frontend, mobile and QA.

In testing you use it to derive test cases, import requests into Postman, generate mock servers, validate responses against the schema, and run spec-driven tools such as Schemathesis or Dredd. You also check that the documentation matches real behaviour.

Real-time e-commerce example

The spec says POST /orders returns 201 and 400, but the real API returns 200 on success and 500 on a bad coupon. Both are defects: one in the implementation and one in the documentation.

How to work on it

- Import openapi.json into Postman and run every documented response code.
- Report any mismatch between the spec and real behaviour as a defect.
- Try Schemathesis against the spec on a test environment.

Section B: Functional Test Design (E-commerce Scenarios)

Q11. How do you design test cases for the Create Order API (POST /orders)?

Answer

Break it into groups.

Positive: single item, multiple items, with coupon, each payment method, guest versus logged-in user. Verify 201, orderId, totals, status CREATED, stock reduced and a database row created.

Negative: no token, expired token, empty items, unknown or inactive product, quantity 0, -1 or text, out of stock, invalid address or pincode, missing required fields, wrong data types, expired coupon, duplicate submit.

Business rules: tax, shipping charges, and the price must come from the server, not from the client.

Non-functional: response time, concurrency and idempotency.

```
{
  "items": [
    { "productId": 101, "quantity": 2 },
    { "productId": 205, "quantity": 1 }
  ],
  "addressId": 55,
  "couponCode": "FEST10",
  "paymentMethod": "UPI"
}
```

Real-time e-commerce example

Order with 2 x Rs 500 plus 1 x Rs 799 and a 10 percent coupon: your test calculates the expected total itself (1799 - 179.90 = 1619.10 before tax) and compares it with the response.

How to work on it

- Write the cases in a table: ID, scenario, input, expected status, expected body, expected DB result.
- Cover each group above before you start automating.

Q12. How do you test the Login API? Give positive and negative cases.

Answer

Positive: valid email and password returns 200 with an access token (and refresh token), token works on a protected endpoint.

Negative: wrong password, unknown user, empty fields, wrong data types, very long values, SQL injection text, account locked after N failed attempts, disabled user.

Security: the wrong-password and unknown-user messages should be identical (to prevent user enumeration), the password or hash is never returned, HTTPS only, tokens expire, logout invalidates the session, and the endpoint is rate limited.

Real-time e-commerce example

Ten wrong passwords in a row for the same user should trigger a lockout or a 429, and a correct password right after should be handled according to the lockout policy.

How to work on it

- Create a data table of email and password pairs with the expected status and message.
- Run it in the Postman Collection Runner or as a parameterised automated test.

Q13. How do you use equivalence partitioning and boundary value analysis on API inputs?

Answer

Equivalence partitioning groups inputs that should behave the same, so you test one value per group. Boundary value analysis tests the edges, where most bugs hide.

Field and rule	Invalid	Valid	Boundary values to send
Quantity 1 to 10	less than 1, more than 10, text, null	1 to 10	0, 1, 2, 9, 10, 11
Pincode (6 digits)	5 or 7 digits, letters	6 digits	99999, 100000, 999999, 1000000

Field and rule	Invalid	Valid	Boundary values to send
Coupon code (max 12 chars)	13+ chars, spaces, symbols	1 to 12 chars	0, 1, 12, 13 characters
Price (2 decimals, above 0)	0, negative, 3 decimals	0.01 and up	0.00, 0.01, 0.001

Real-time e-commerce example

Max quantity per order is 10. You test 9, 10 and 11 on POST /cart/items. 10 succeeds, 11 returns a validation error with a clear message.

How to work on it

- For every input field write: type, min, max, required, format.
- Generate the boundary values from that list so no field is missed.

Q14. How do you test the Add to Cart API?

Answer

Cases: add a new item, add the same item again (quantity should increase or merge according to design), update quantity, remove item, stock limit, per-order maximum, inactive or deleted product, price in the cart matches the catalog, cart total recalculates, cart persists after re-login, guest cart merges after login, and two devices with the same account stay in sync.

Real-time e-commerce example

Add product 101 (stock 3) with quantity 5. The API should return a clear stock error (409 or 422) and the cart should stay unchanged.

How to work on it

- Use one test user per test so carts do not clash.
- After every call also call GET /cart and verify totals, not just the add response.

Q15. How do you test search, filter, sort and pagination APIs?

Answer

Example: GET /products?q=shoes&category=men&minPrice=500&maxPrice=2000&sort=price_asc&page=2&size=20.

Check: every result matches the filters, the sort order is truly sorted, total count is correct, pages do not repeat items, the last page works, a page beyond the last returns an empty list (not 500), size 0 or -1 is rejected, a huge size is capped, an empty result returns 200 with an empty array, special characters are handled, and combined filters work.

```
const prices = pm.response.json().items.map(p => p.price);
const sorted = [...prices].sort((a, b) => a - b);
pm.test("Sorted by price ascending", () => pm.expect(prices).to.eql(sorted));
pm.test("Within price range", () =>
  pm.expect(prices.every(p => p >= 500 && p <= 2000)).to.be.true);
```

Real-time e-commerce example

Call page=1 and page=2 and assert that no product ID appears in both lists.

How to work on it

- Collect product IDs from all pages in a loop and check for duplicates and for the total count.
- Test with a large catalog and measure response time.

Q16. How do you test coupon, discount and price calculation APIs?

Answer

Cases: percentage and flat coupons, expired, not yet started, minimum cart value, maximum discount cap, one use per user, stacking rules, category-specific coupons, case sensitivity, removing a coupon, removing items after applying a coupon, rounding to 2 decimals, and tax calculated after discount.

Never copy the application's result as the expected value. Calculate it independently in the test. Use decimals, not floating point, because 0.1 + 0.2 is not exactly 0.3.

Real-time e-commerce example

Cart Rs 2,000, coupon FEST10 (10 percent, max discount Rs 150, minimum cart Rs 1,500). Expected discount is Rs 150 (not 200 because of the cap).

How to work on it

- Build a data table of cart value, coupon, expected discount and expected total.
- Run it data-driven, then add concurrency tests for one-time coupons.

Q17. How do you validate an API response beyond the status code?

Answer

Check the schema (required fields, data types, enums), mandatory fields, null versus missing fields, formats (ISO 8601 dates, currency with 2 decimals), values against expected data or the database, array sizes, headers, response time, and that no extra sensitive fields are returned.

```
const schema = {
  type: "object",
  required: ["orderId", "status", "total"],
  properties: {
    orderId: { type: "string" },
    status: { type: "string", enum: ["CREATED", "PAID", "SHIPPED"] },
    total: { type: "number" }
  }
};
pm.test("Schema is valid", () => pm.response.to.have.jsonSchema(schema));
```

Real-time e-commerce example

A product API returns price as the string '499.00' instead of the number 499. The UI works today, but the mobile app crashes after a release. A schema test catches this immediately.

How to work on it

- Keep one JSON schema file per response and reuse it in tests.
- In Java use the json-schema-validator module of REST Assured.

Q18. How do you test a Payment API?

Answer

Always use the gateway sandbox and its test cards, never real cards.

Cases: success, declined card, insufficient funds, expired card, wrong CVV, 3-D Secure or OTP flow, gateway timeout, gateway down, duplicate payment click, amount or currency mismatch, partial and full refund, and webhook callbacks with valid and invalid signatures.

Also check: the order is not marked PAID after a failure, no card data appears in responses or logs, only a masked card is shown, and the CVV is never stored.

Real-time e-commerce example

The gateway times out after the user is charged. The order must end in a clear pending or failed state and be reconciled later, not stay stuck as CREATED while the money is gone.

How to work on it

- Use a mock payment server (WireMock) to simulate declines and slow responses.
- Run a few real sandbox tests for the happy path and refund.

Q19. How do you test chained or dependent API calls (an end-to-end flow)?

Answer

Flow: login, search, add to cart, apply coupon, create order, pay, get order, cancel or return. Extract values from one response (token, IDs) and use them in the next request.

Keep each test independent: create its own data through APIs in setup, and clean it afterwards, so one failure does not break the rest.

```
// Postman: Tests tab of the Login request
const body = pm.response.json();
pm.environment.set("token", body.accessToken);
// Later requests use the header:  Authorization: Bearer {{token}}
```

Real-time e-commerce example

After POST /orders, store orderId and call GET /orders/{{orderId}} to confirm the status, then PATCH /orders/{{orderId}}/cancel and confirm the status is CANCELLED and the stock is restored.

How to work on it

- Build one Postman collection with folders for each flow.
- In code frameworks, write helper methods such as createUser(), addToCart() and placeOrder().

Q20. What is idempotency, and how do you test it for orders and payments?

Answer

An idempotent operation gives the same result when repeated. POST is not idempotent by default, so critical POST calls use an Idempotency-Key header: the server saves the first response and returns it for repeated requests with the same key.

Test: send the same POST /orders with the same key twice and expect one order and the same response. Use a different key and expect a new order. Retry after a timeout. Send two identical requests in parallel.

Real-time e-commerce example

A mobile user taps Pay, the network drops, and the app retries. With an idempotency key the customer is charged once. Without it, the customer is charged twice.

How to work on it

- Check the database after the test: exactly one order and one payment row.
- Use k6 or a thread pool to fire the duplicate requests at the same moment.

Section C: Security and Authentication

Q21. What is the difference between authentication and authorization? Which authentication types have you tested?

Answer

Authentication proves who you are. Authorization decides what you may do.

Types: Basic (base64 user:password, only safe over HTTPS), API key, Bearer token / JWT, OAuth 2.0 (authorization code for users, client credentials for partner systems), and session cookies.

Test each with: no credentials, wrong credentials, expired credentials, valid credentials with the wrong role, and credentials from another environment.

Real-time e-commerce example

A customer token calling POST /admin/coupons should return 403. A request with no token should return 401.

How to work on it

- Create test users for each role: customer, seller, admin.
- Run every endpoint against every role and record the expected status in a matrix.

Q22. How do you test JWT-based authentication?

Answer

A JWT has three base64url parts: header, payload and signature. The payload is only encoded, not encrypted, so it must not contain sensitive data.

Tests: expired token (exp) returns 401, missing token, tampered payload (change role to admin but keep the old signature) returns 401, the 'alg: none' attack, signature made with a different key, wrong audience or issuer, refresh token flow, token reuse after logout, and a sensible token lifetime.

Real-time e-commerce example

Take a customer token, edit the payload to role=ADMIN, and call GET /admin/orders. The signature check must fail and the API must return 401.

How to work on it

- Use jwt.io to read a token from a test environment (never paste production tokens into online tools).
- Add a Postman pre-request script to create the modified tokens.

Q23. What is BOLA / IDOR, and how do you test for it?

Answer

Broken Object Level Authorization (also called IDOR) happens when the API trusts an ID from the client without checking who owns the object. It is one of the most common and serious API flaws.

Test with two accounts. Create data as user A, then call the same endpoint with user B's token. Also change IDs in the path, query and body. Check list endpoints return only the caller's data.

Real-time e-commerce example

User A (ID 101) places order 9002. User B calls GET /orders/9002, GET /invoices/9002 and PUT /users/101/address. All three must return 403 or 404.

How to work on it

- Keep two or more test users in your framework.
- For every endpoint that has an ID, add one cross-user test.

Q24. What is the OWASP API Security Top 10 and how do you apply it?

Answer

It is a list of the most critical API risks. The list below is the 2023 edition. Check owasp.org for the current version before the interview.

Risk	E-commerce test example
API1 Broken Object Level Authorization	Read another customer's order by changing the ID
API2 Broken Authentication	No lockout, weak or unsigned tokens, long-lived tokens
API3 Broken Object Property Level Authorization	Response leaks internal fields, or the client can set price or isAdmin
API4 Unrestricted Resource Consumption	No rate limit or page-size cap, huge requests slow the system
API5 Broken Function Level Authorization	Customer token can call admin APIs (create coupon, refund)
API6 Unrestricted Access to Sensitive Business Flows	Bots buy limited stock, coupon abuse, mass sign-ups
API7 Server Side Request Forgery	Image-by-URL upload fetches internal addresses
API8 Security Misconfiguration	Verbose errors, CORS open to all, debug endpoints on
API9 Improper Inventory Management	Old /v1 or test endpoints still live and unprotected
API10 Unsafe Consumption of APIs	App trusts third-party responses without validating them

Real-time e-commerce example

You review the list and turn each risk into at least one test case for the cart, order and payment APIs.

How to work on it

- Make a checklist from this table and run it on each release.
- Run OWASP ZAP or Burp Suite (community edition) on a test environment you are authorised to scan.

Q25. How do you test for injection attacks (SQL injection, XSS) in APIs?

Answer

Send attack strings into every input: query, path, body and headers. Examples: ' OR '1'='1, a stray quote, and <script>alert(1)</script> in a product review.

Expected result: the input is rejected (400 or 422) or stored safely and escaped on output. There must be no database error text, no 500, and no script execution when the data is shown later.

Only test on environments you are authorised to test.

Real-time e-commerce example

POST /products/101/reviews with the text <script>alert(1)</script>. When the review is listed, the script must appear as plain text and never run.

How to work on it

- Build a small payload list and run it through the Postman Collection Runner.
- Use OWASP ZAP active scan on QA for broader coverage.

Q26. How do you test rate limiting and brute-force protection?

Answer

Send many requests quickly to login, OTP, coupon and search endpoints. Expect 429 Too Many Requests with a Retry-After header, or an account lockout. Limits may apply per user, IP or API key. After the window passes the limit must reset, and normal users must not be affected.

Real-time e-commerce example

Request an OTP 20 times in one minute for the same mobile number. The API should stop after the limit, otherwise attackers can flood SMS or guess codes.

How to work on it

- Use k6 or the Postman Runner with a loop to send the burst.
- Verify the 429 body and the Retry-After header.

Q27. How do you check for sensitive data exposure?

Answer

Responses must not contain password hashes, full card numbers, CVV, internal IDs or other users' personal data. Cards must be masked (for example **** 1111). Use HTTPS only and check that plain HTTP is blocked or redirected.

Tokens must not be placed in URLs, because URLs end up in logs. Error messages must be generic, with no stack traces or server versions. Logs must mask sensitive values.

Real-time e-commerce example

GET /users/me should return only the allowed fields. If the response contains 'passwordHash' or the full card number, it is a high-severity defect.

How to work on it

- Compare each response with an approved list of fields.
- Search the response and application logs for card numbers, passwords and tokens.

Q28. What is mass assignment and price tampering? How do you test business logic abuse?

Answer

Mass assignment happens when the API accepts fields the client should never control. Price tampering is the same idea applied to money.

Test by adding extra fields: 'price': 1, 'discount': 100, 'role': 'ADMIN', 'isPaid': true or another user's userId. The server must ignore or reject them and use server-side values.

Also try: negative quantity to create a refund, applying a coupon many times, reusing a one-time coupon in parallel, skipping the payment step by calling 'confirm order' directly, and cancelling an order after delivery.

```
POST /api/v1/orders
{ "items": [{ "productId": 101, "quantity": 1, "price": 1 }],
  "isPaid": true, "discount": 100 }
// Expected: price and flags ignored or rejected. Total uses the catalog price.
```

Real-time e-commerce example

A phone costs Rs 49,999 but the order payload carries price 1. If the order total becomes Rs 1, that is a critical defect.

How to work on it

- Add 'extra field' tests to every create and update API.
- Draw the order status flow and test every illegal transition.

Section D: Data, Integration and Async Flows

Q29. How do you validate API behaviour against the database?

Answer

After a write call, query the database and compare. Check rows created or updated, totals, status, timestamps, foreign keys and inventory changes. After a failed call confirm nothing was saved (rollback). After a delete check whether it is a hard delete or a soft delete flag.

In automation, connect with JDBC (Java) or a Python driver and run the query inside the test.

```
SELECT o.order_id, o.status, o.total_amount, COUNT(oi.item_id) AS items
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_id = 'ORD-9002'
GROUP BY o.order_id, o.status, o.total_amount;
```

Real-time e-commerce example

POST /orders returns 201. The database must show one order row with status CREATED, the correct item rows, and the product stock reduced by the ordered quantity.

How to work on it

- Get a read-only database user for the QA environment.
- Write the verification queries first, then add them to the automated tests.

Q30. How do you manage test data in API testing?

Answer

Create data through APIs in setup, use unique values (timestamped emails), and clean up in teardown. Keep dedicated test accounts and seeded products with large stock. Avoid shared data that tests change. Use data builders or factories, and make tests safe for parallel runs. Mask production data if it is copied to lower environments.

Real-time e-commerce example

Each test registers its own user with an email like qa_20261108_153012@test.com, so two parallel runs never collide and no real customer is touched.

How to work on it

- Add a TestDataFactory with methods such as createUser(), createProduct() and createOrder().
- Add a cleanup script that deletes records tagged 'qa_'.

Q31. How do you test integrations with third-party services such as payment, shipping, SMS and email?

Answer

Use the vendor's sandbox where available. For everything else use mocks, stubs or service virtualization (WireMock, MockServer, Mockoon, Postman mock servers). Mocks let you create failures on demand (timeouts, 500, malformed data), avoid cost and rate limits, and keep tests stable.

Still run a few real sandbox tests for the happy path, because mocks can drift away from the real service.

Real-time e-commerce example

Mock the courier API to return 'Delivered', 'Delayed' and a 500 error, and confirm the order status API shows the right state in each case.

How to work on it

- Record real sandbox responses and replay them as mock responses.
- Add contract tests so the mock stays accurate.

Q32. How do you test webhooks and asynchronous or event-driven flows?

Answer

Webhook example: the payment gateway calls POST /webhooks/payment. Test a valid signature (order becomes PAID), an invalid signature (rejected), duplicate delivery (processed once), out-of-order events, delayed delivery and retries after a 500.

For queues (Kafka, RabbitMQ) check that the right event is published after an action and that its payload is correct. Because results arrive later, poll with a timeout instead of using fixed sleeps.

Real-time e-commerce example

After payment succeeds, an OrderPaid event must be published once, and the notification service must send one SMS, not two.

How to work on it

- Use webhook.site or ngrok to inspect callbacks during manual testing.
- Write a helper waitUntil(condition, timeout) for eventual-consistency checks.

Q33. How do you test file upload and download APIs?

Answer

Upload (multipart/form-data, for example a product image): allowed types (jpg, png), blocked types (exe, php), size limit, zero-byte file, double extensions (image.jpg.php), unsafe file names, content type that does not match the real file, and virus scanning.

Download (for example an invoice PDF): correct Content-Type and Content-Disposition, the file opens, content matches the order, authorization (nobody can download someone else's invoice), and speed for large files.

Real-time e-commerce example

GET /invoices/9002 with user B's token must not return user A's invoice PDF.

How to work on it

- In Postman use Body > form-data and set the key type to File.
- Keep a small folder of valid and invalid sample files.

Q34. How do you test API versioning and backward compatibility?

Answer

Adding a field is usually safe. Removing, renaming or changing the type of a field breaks existing clients. Old mobile app versions stay in use for months, so v1 must keep working when v2 is released.

Run the v1 regression suite against every new build, check deprecation headers, and use an OpenAPI diff tool to flag breaking changes.

Real-time e-commerce example

Backend renames 'productName' to 'name'. The new app works, but the old app shows blank product titles. A v1 regression test would have caught it.

How to work on it

- Tag tests by API version.
- Add an OpenAPI diff step to the pipeline.

Q35. How do you test concurrency and race conditions, for example the last item in stock?

Answer

Send simultaneous requests and check the final state. Only one of two parallel orders for the last unit should succeed, the other should get 409 or an out-of-stock error, and stock must never go below zero. The same applies to a one-time coupon and to double-click payments.

Developers use locks or optimistic concurrency (ETag with If-Match, returning 412 on a mismatch). QA verifies the outcome in the database.

Real-time e-commerce example

Product 205 has stock 1. Two users call POST /orders at the same moment. Expected: one 201, one 409, stock 0.

How to work on it

- Use k6, JMeter, or a thread pool in your own framework to send parallel calls.
- Repeat the test many times, because race conditions do not fail every run.

Section E: API Automation and CI/CD

Q36. Which API testing tools have you used, and how do you choose between them?

Answer

Choose by team skills, CI fit, maintainability and cost.

Tool	Best for
Postman / Bruno	Exploration, quick collections, sharing with the team
REST Assured (Java)	Large regression suites in a Java team, with TestNG or JUnit
Karate	Readable BDD-style tests with little code, built-in reports
pytest + requests (Python)	Python teams, flexible data handling
Playwright API / Supertest	JavaScript or TypeScript teams, mixing API and UI tests
SoapUI	SOAP and WSDL services
k6 / JMeter / Gatling	Performance and load testing
OWASP ZAP / Burp Suite	Security scanning
Pact	Consumer-driven contract testing
WireMock / Mockoon	Mocking third-party services

Real-time e-commerce example

A Java team with an existing TestNG and Selenium framework picks REST Assured. A manual-testing team starting automation picks Postman or Karate.

How to work on it

- Be ready to explain why you chose a tool, not only how to use it.
- Know at least one code-based tool and Postman well.

Q37. How do you design an API automation framework?

Answer

Use layers: configuration (environments), a base API client (request spec, auth, logging), service classes per area (the API version of page objects), models or builders for payloads, test data factories, test classes, utilities (schema validation, assertions), reporting (Allure or HTML) and CI.

Principles: no hard-coded URLs or credentials, reusable methods, independent tests, tags for smoke and regression, parallel execution, and clear failure messages.

```
api-tests/
  config/        env.qa.json, env.stage.json
  clients/       BaseClient, AuthClient
  services/      CartApi, OrderApi, PaymentApi
  models/        OrderRequest, OrderResponse
  testdata/      users.json, coupons.csv, TestDataFactory
  schemas/       order.schema.json
  tests/         smoke/, regression/, security/
  utils/         DbHelper, Assertions, WaitUtils
  reports/
```

Real-time e-commerce example

OrderApi.createOrder(token, payload) is written once and used by 30 tests. When the endpoint path changes, you fix one file.

How to work on it

- Start small: one client class and two service classes, then grow.
- Add logging of request and response (with secrets masked) so failures are easy to read.

Q38. How do you write assertions and scripts in Postman? What is Newman?

Answer

The Pre-request Script tab runs before the call (create data, build tokens). The Tests tab runs after the response (assertions, saving variables). Use environment variables for URLs and tokens. Newman (or the Postman CLI) runs collections from the command line and in CI.

```
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.test("Order created", () => {
  const body = pm.response.json();
  pm.expect(body.orderId).to.be.a("string");
  pm.expect(body.status).to.eql("CREATED");
  pm.environment.set("orderId", body.orderId);
});
pm.test("Response time under 800 ms", () =>
  pm.expect(pm.response.responseTime).to.be.below(800));
```

Real-time e-commerce example

The Login request saves the token, the Create Order request uses it, and the Get Order request uses the saved orderId.

How to work on it

- Export the collection and environment to files.
- Run: newman run orders.postman_collection.json -e qa.postman_environment.json

Q39. Show a REST Assured example, including token handling.

Answer

REST Assured reads like plain English with given, when and then. Reuse a RequestSpecification for base URL and headers, and extract values from one response to use in the next.

```
String token = given().contentType(ContentType.JSON)
    .body(Map.of("email", "qa.user@test.com", "password", "Test@123"))
  .when().post("/auth/login")
  .then().statusCode(200)
    .extract().path("accessToken");

given().header("Authorization", "Bearer " + token)
```

```
    .contentType(ContentType.JSON)
    .body(orderPayload)
  .when().post("/orders")
  .then().statusCode(201)
    .body("status", equalTo("CREATED"))
    .body("items.size()", greaterThan(0))
    .time(lessThan(1500L));
```

Real-time e-commerce example

The test logs in, creates an order, and then checks the database to confirm the same order exists.

How to work on it

- Practise on a public API such as reqres.in or a demo shop API.
- Add JSON schema validation and POJO serialization once the basics are comfortable.

Q40. How do you integrate API tests into CI/CD?

Answer

Run a small smoke suite (2 to 5 minutes) on every pull request. Run the regression suite after each deployment to QA or staging. Run the full suite and performance checks nightly. Fail the pipeline on test failure, publish reports, run tests in parallel, and read URLs and credentials from CI variables and secrets.

Typical commands: newman for Postman, mvn test with a tag for REST Assured, pytest -m smoke for Python.

```
newman run shop.postman_collection.json -e qa.postman_environment.json \
       --reporters cli,junit --reporter-junit-export results.xml

mvn test -Dgroups=smoke -DbaseUrl=$QA_BASE_URL
```

Real-time e-commerce example

A developer opens a pull request that changes the cart service. The smoke suite runs automatically, finds that POST /cart/items now returns 500 for guests, and blocks the merge.

How to work on it

- Set up a free Jenkins or GitHub Actions pipeline for a practice project.
- Keep smoke tests fast and stable, otherwise the team will ignore them.

Q41. How do you implement data-driven API testing?

Answer

Keep inputs and expected results in CSV, JSON, Excel or a database, and run one test for every row. Use the Collection Runner in Postman, DataProvider (TestNG), parameterised tests (JUnit 5) or pytest.mark.parametrize.

```
@pytest.mark.parametrize("qty, expected", [(0, 400), (1, 201), (10, 201),
                                           (11, 400), (-1, 400)])
def test_add_to_cart_quantity(api, qty, expected):
    r = api.post("/cart/items", json={"productId": 101, "quantity": qty})
    assert r.status_code == expected   # adjust 201 to your API's real code
```

Real-time e-commerce example

A CSV with 30 coupon codes, cart values and expected discounts runs as 30 test cases from one script.

How to work on it

- Store test data outside the test code so testers can add rows without coding.
- Print the row number or input in the failure message.

Q42. How do you handle environments and secrets in API tests?

Answer

Keep base URL and credentials in environment files or CI variables, never in code. Use Postman environments (secret type), .env files that are git-ignored, or a vault. Use separate test accounts per environment. Do not run write tests against production. Production checks, if any, should be read-only smoke tests agreed with the team.

Real-time e-commerce example

The same suite runs against QA with -DbaseUrl=https://qa.shop.example.com and against staging with a different value, with no code change.

How to work on it

- Search your repository for hard-coded tokens and URLs and move them to config.
- Add .env to .gitignore.

Q43. Your API tests are flaky. How do you find the cause and fix it?

Answer

Common causes: shared or reused data, test order dependency, timing and async delays, unstable environments, third-party calls, rate limits, and assertions on dynamic values such as timestamps or generated IDs.

Fixes: independent data per test, polling with a timeout instead of fixed sleeps, assert stable fields only, mock unstable third parties, retry only network-level failures (retries must not hide real bugs), quarantine and fix flaky tests quickly, and track the flaky rate.

Real-time e-commerce example

A test checks an order status immediately after payment, but the status updates through an async event. It passes 7 times out of 10. Replace the immediate check with a poll for up to 10 seconds.

How to work on it

- Run the failing test 20 times in a loop to confirm it is flaky.
- Check the logs for the failing runs and compare them with the passing ones.

Section F: Performance and Resilience

Q44. How do you do performance testing of APIs? Which metrics matter?

Answer

Metrics: response time (average, p90, p95, p99), throughput (requests per second), error rate, concurrent users, and server resources (CPU, memory, database connections).

Steps: pick the critical flows, build a workload model from real traffic, run a baseline, test under load, find the bottleneck, tune, and retest. Set targets first, for example p95 under 800 ms and an error rate under 1 percent. Treat these numbers as examples and agree real ones with the team.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 100, duration: '5m',
  thresholds: { http_req_duration: ['p(95)<800'], http_req_failed: ['rate<0.01'] },
};
export default function () {
  const res = http.get('https://qa.shop.example.com/api/products?page=1');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Real-time e-commerce example

Before a festival sale, thousands of users search, open products, add to cart and check out at the same time. You model that mix and find that GET /products slows down because of a missing database index.

How to work on it

- Install k6 and run this script against a demo API.
- Learn to read percentiles, not just averages.

Q45. What are the types of performance testing, with e-commerce examples?

Answer

Different tests answer different questions.

Type	Goal	E-commerce example
Load	Behaviour at expected traffic	5,000 users shopping on a normal day
Stress	Find the breaking point	Increase users until errors appear
Spike	Sudden jump in traffic	Flash sale starts at 12:00 and traffic multiplies in seconds
Soak / endurance	Stability over hours	Run 8 hours to find memory leaks
Scalability	Does adding servers help?	Compare results at 2, 4 and 8 instances
Volume	Large data sizes	Search over 5 million products

Real-time e-commerce example

Spike testing matters most for sales events, because auto-scaling often reacts too slowly to a sudden jump.

How to work on it

- Pick one scenario per type and write the expected result before you run it.

Q46. How do you test resilience: timeouts, retries, circuit breakers and downstream failures?

Answer

Simulate failures and check graceful behaviour. If the payment gateway is slow, the order service should time out cleanly (504 or a clear error) and must not mark the order paid. If a downstream service returns 500, retries should be limited with back-off and must not create duplicates. If the database is down, return 503 with no stack trace. If the recommendation service fails, the product page should still load.

Tools: WireMock delays and faults, Toxiproxy, and basic chaos testing.

Real-time e-commerce example

Make the inventory service return 500. The cart API should respond with a clear error and keep the existing cart intact.

How to work on it

- Add one failure-simulation test for each external dependency.
- Check the logs and monitoring alerts during the test.

Q47. How do you test caching behaviour in APIs?

Answer

Check Cache-Control, ETag and If-None-Match (304 Not Modified), CDN behaviour, and cache invalidation. Private data such as the cart and orders must never be cached publicly (look for Cache-Control: private or no-store). Changes such as price and stock must appear after the cache time-to-live.

Real-time e-commerce example

A price changes from Rs 999 to Rs 899 in the admin tool. After the cache expires, GET /products/101 must return 899, and the cart must never show another user's items.

How to work on it

- Call the same endpoint twice and compare headers and response times.
- Test the same endpoint with two different users to catch leaked cached data.

Section G: Advanced and Lead-Level Questions

Q48. What is contract testing, and how does it help in a microservices e-commerce system?

Answer

Contract testing checks that a service provider and its consumers agree on the request and response format, without running a full integration environment. In consumer-driven contract testing (Pact), the consumer records its expectations and the provider verifies them in CI.

It catches breaking changes before deployment and runs faster than full integration tests.

Real-time e-commerce example

The Order service consumes the Inventory service. The Inventory team renames 'available' to 'stock'. The Pact verification fails in their pipeline, and the change is fixed before it breaks checkout.

How to work on it

- Try the Pact tutorial with two small services.
- Learn what the Pact Broker and can-i-deploy do.

Q49. An API test fails in the QA environment. How do you debug it, and how do you write a good defect report?

Answer

Reproduce it, then isolate: is it the request, the test data, the environment or the code? Check the exact request and response, headers and token, the logs using the correlation or trace ID (Kibana, Splunk, Datadog), the database state, recent deployments, and compare with another environment.

A good defect report has: a clear title, environment and build, endpoint and method, the request (as a curl command, with secrets removed), expected versus actual result, the response and status code, trace ID, how often it happens, severity and attachments.

```
Title: POST /orders returns 500 when couponCode has lowercase letters
Env/Build: QA, build 2026.11.08.3
Request: curl -X POST https://qa.shop.example.com/api/v1/orders -H 'Authorization:
  Bearer <masked>' -d '{"items":[{"productId":101,"quantity":1}],"couponCode":"fest10"}'
Expected: 201 with discount, or 422 COUPON_INVALID
Actual: 500 Internal Server Error, traceId a1b2c3
Frequency: 5 out of 5. Severity: High (checkout blocked)
```

Real-time e-commerce example

The trace ID in the response lets the developer find the NullPointerException in the logs in a minute instead of an hour.

How to work on it

- Learn the Postman Console, browser DevTools Network tab and a proxy such as Charles or Fiddler.
- Always ask the developers for the trace ID or log link.

Q50. As a QA lead, how would you define an API test strategy for a new e-commerce release?

Answer

1. Understand the scope and risks with product and developers. 2. Review the OpenAPI spec and join design reviews (shift-left). 3. Prioritise by risk. 4. Choose test types: functional, security, performance, contract and regression. 5. Plan data and environments. 6. Decide automation scope and CI stages. 7. Define entry and exit criteria. 8. Track metrics and report. 9. Review, mentor and improve.

Priority	Areas	Depth
Critical	Login, payment, order, refund	Functional, security, concurrency, performance, contract
High	Cart, inventory, coupons, address	Functional, boundary, negative, regression
Medium	Search, product details, profile	Functional, pagination, performance
Low	Wishlist, reviews, recommendations	Smoke and basic negative

Real-time e-commerce example

Example exit criteria: all smoke and critical tests pass, no open critical or high defects, p95 response time within the agreed target, and no high-severity security findings. Example metrics: endpoint coverage, pass rate, defect leakage to production, flaky rate and suite run time.

How to work on it

- Prepare a one-page strategy for your current or a practice project and be ready to explain every choice.
- Be ready to discuss trade-offs: what you would not automate and why.

Extra 1: Universal API test checklist

Use this list for any endpoint, in an interview or on the job. If the interviewer gives you an unfamiliar API, walk through it in this order.

- Status code: correct for success and for every error case.
- Response body: schema, data types, mandatory fields, values, nulls, formats.
- Headers: Content-Type, caching, security headers, Location.
- Authentication and authorization: no token, expired, wrong role, other user's data.
- Input validation: null, empty, wrong type, length, boundary, special characters, injection text.
- Business rules: totals, tax, discounts, stock, status transitions.
- Database and side effects: rows, inventory, events, emails, rollback on failure.
- Idempotency and duplicates: the same request sent twice or in parallel.
- Error handling: clear error code and message, no stack traces, no 500 for client mistakes.
- Performance: response time and behaviour under load.
- Security: sensitive data, rate limits, mass assignment, OWASP checks.
- Integration and resilience: third-party failures, timeouts, retries.
- Compatibility: older API versions and backward compatibility.

Extra 2: How to answer any question

1. **Define** the concept in one or two sentences.
2. **Give an e-commerce example** (cart, order, payment, coupon, inventory).
3. **Explain how you tested it**: the cases, tools and checks.
4. **Mention a result or a lesson**: a bug you found or how you improved the process.

Extra 3: 60-second project pitch (template)

Replace the items in [brackets] with the facts from your own project. Do not claim tools or results you have not actually used.

> I work on an e-commerce platform built from services for catalog, cart, order, payment and inventory. I own API testing for [cart / checkout / payments]. I design test cases from the OpenAPI spec using equivalence partitioning, boundary values and negative cases. I automate with [Postman + Newman / REST Assured / Karate] and run the suites in [Jenkins / GitHub Actions]. I validate data in the database, test security cases such as authorization and rate limiting, and support performance tests with [k6 / JMeter]. A defect I am proud of: [describe a real bug you found, its impact and how you reported it].

Extra 4: Common follow-up questions to prepare

- Which API bug had the biggest impact, and how did you find it?
- How do you decide what to automate and what to test manually?
- How do you test an API that has no documentation?
- How do you handle a developer who disagrees that something is a bug?
- How do you keep test data clean when many testers share one environment?
- What would you do if a critical API test fails one hour before release?